Memory Management & Garbage Collection

13 min read · Updated 17 Jul 2026 · Topic 23 of 23 · Bookmark · Completed

JavaScript manages memory for you: you allocate by creating values, and the engine frees memory automatically when values become unreachable. There is no `malloc` / `free`. Interviewers care about this topic because "automatic" does not mean "leak-proof" — the most common production bugs in long-lived front-ends (SPAs, dashboards, Node servers) are memory leaks caused by accidentally keeping references alive. Knowing *how* the collector decides what to keep is what lets you reason about, and fix, those leaks.

Stack vs heap

Memory is conceptually split into two regions. The **stack** holds primitive values (`number`, `string`, `boolean`, `null`, `undefined`, `symbol`, `bigint`) and the *references* (pointers) to objects. It is fast, fixed-size per frame, and LIFO. The **heap** holds objects, arrays, and functions — dynamically sized structures. A variable that "holds an object" actually holds a reference on the stack that points at the object on the heap.

This is why primitives copy by value and objects copy by reference:

JS · Copy

```
let a = 10;
let b = a;       // the value 10 is copied
b = 20;
console.log(a, b); // ⇒ 10 20

const o1 = { n: 1 };
const o2 = o1;   // the reference is copied — same heap object
```

`a` and `b` are independent because each holds its own primitive. `o1` and `o2` are two references to *one* heap object, so mutating through one is visible through the other.

Reachability: the one rule that matters

The garbage collector's whole job reduces to a single question: **is this value reachable?** A value is kept in memory if and only if it can be reached, by following references, from a set of **roots**. Everything else is garbage and may be freed.

The roots are the things the engine treats as inherently "alive":

- The **global object** (`globalThis` / `window`) and everything it references.
- The **currently executing call stack** — local variables and parameters of functions in progress.
- **Closures** — variables captured by functions that are themselves reachable.

Reachability is transitive: if a root references object A, and A references B, and B references C, then all three are alive. The moment nothing reachable points at an object, it becomes eligible for collection — even if that object internally still points at other things.

Mark-and-sweep: the real algorithm

Modern engines (V8 in Chrome/Node, SpiderMonkey in Firefox, JavaScriptCore in Safari) use **mark-and-sweep**, not reference counting. It runs in two phases:

1. **Mark** — start from the roots and traverse every reference, marking each reachable object as "live". This is a graph traversal (the object graph), so anything reachable by any path gets marked.
2. **Sweep** — walk the heap; any object *not* marked is unreachable, so its memory is reclaimed.

Because marking works from roots outward, **it does not care about cycles**. Two objects that point at each other but that nothing reachable points *to* are simply never marked,

so they are collected. This is the key advantage over reference counting.

Real engines add sophistication on top of this — V8 uses a **generational** collector (a small, frequently-collected "young generation" for short-lived objects, and an "old generation" for survivors), does **incremental/concurrent** marking to avoid long pauses, and compacts memory — but the mental model interviewers want is plain mark-and-sweep from roots.

Why reference counting alone fails

An older, simpler scheme keeps a count of how many references point at each object and frees it when the count hits zero. It has a fatal flaw: **cycles**. Two objects referencing each other keep each other's count at 1 forever, even when the rest of the program can no longer reach them.

JS Copy

```
function makeCycle() {
  const x = {};
  const y = {};
  x.other = y;
  y.other = x;   // x and y reference each other
  return null;   // nothing outside can reach them anymore
}
makeCycle();
```

After `makeCycle()` returns, `x` and `y` are unreachable from any root, but each still has an incoming reference (from the other), so their counts never reach zero. A pure reference-counting collector would leak them. Mark-and-sweep collects them, because neither is reachable from a root.

Common memory leaks (and how to fix each)

A leak in JS is not a dangling pointer — it is a reference you *forgot to drop*, keeping an object reachable long after you are done with it. Here are the classic ones interviewers ask you to name.

1. Lingering globals

JS Copy

```
function process(data) {
  results = data.map(x => x * 2); // BUG: missing `let` → global `results`
  return results;
}
```

The missing declaration attaches `results` to the global object, so it (and whatever `data` was) survives for the life of the app. **Fix:** always declare (`const` / `let`), and use `'use strict'` / modules so an undeclared assignment throws instead of silently leaking.

2. Forgotten timers and intervals

A live `setInterval` keeps its callback — and everything the callback's closure captures — reachable indefinitely.

JS Copy

```
function startPolling(bigBuffer) {
  const id = setInterval(() => {
    read(bigBuffer); // closure retains bigBuffer forever
  }, 1000);
  return id; // must be cleared later
}
const id = startPolling(new Array(1e6));
clearInterval(id); // FIX: stop the timer when done
```

Until you `clearInterval`, the interval is a root-reachable job holding `bigBuffer` alive. **Fix:** always pair `setInterval` / `setTimeout` with a `clear*`, especially on component unmount / route change.

3. Detached DOM nodes

You remove a node from the document, but a JS variable (or an array, or a closure) still references it. The node — and its entire subtree — cannot be collected because JS

keeps it reachable, even though it is no longer in the DOM tree. This is one of the most common SPA leaks.

JS Copy

```
// Illustrative (browser):
const cache = [];
function cacheRow() {
  const row = document.createElement('div');
  document.body.appendChild(row);
  cache.push(row);              // strong ref held in `cache`
  document.body.removeChild(row); // removed from DOM, but NOT freed
}
```

The removed `row` is "detached" but retained by `cache`. **Fix:** drop the references when you remove the node (`cache.length = 0`, `null` out fields), or hold DOM references in a `WeakMap` / `WeakSet` so removal alone lets them go.

4. Closures that capture large data

A closure keeps the *whole* variable environment of its enclosing scope reachable, not just the parts it uses. Capture a giant array to return a function that needs one number, and the array stays alive as long as the function does.

JS Copy

```
function makeHandler() {
  const huge = new Array(1000).fill('x');
  return function () { return huge.length; }; // retains `huge`
}
const handler = makeHandler();
console.log(handler()); // ⇒ 1000  (huge is alive for handler's lifetime)
```

Fix: capture only what you need, so the large object can be collected:

JS Copy

```
function makeHandlerLean() {
  const huge = new Array(1000).fill('x');
```

```
const lean = makeHandlerLean();
console.log(lean()); // ⇒ 1000
```

5. Ever-growing caches

A plain `Map`/object used as a cache never evicts on its own — every key you add is a strong reference the collector must honor.

JS Copy

```
const cache = new Map();
function memoize(key, value) {
  cache.set(key, value); // grows forever, nothing is ever removed
  return value;
}
memoize('a', 1);
console.log(cache.size); // ⇒ 1  (and only ever climbs)
```

Fix: bound the cache — an LRU with a max size and eviction, a TTL, or a `WeakMap` when the key is an object whose lifetime should drive eviction (see below).

6. Event listeners never removed

`addEventListener` holds a strong reference to the handler (and its closure). Add listeners without removing them — on repeatedly-mounted components — and both handlers and captured state accumulate.

JS Copy

```
const bus = {
  fns: [],
  on(f) { this.fns.push(f); },
  off(f) { this.fns = this.fns.filter(x ⇒ x ≠= f); },
};
const onTick = () ⇒ {};
bus.on(onTick);
console.log(bus.fns.length); // ⇒ 1
```

Note the trap: you can only remove a listener you can name. `addEventListener('x', () => {})` with an anonymous handler can never be removed. **Fix:** keep a named reference and remove it on teardown (React's `useEffect` cleanup, `AbortController`'s `signal` for a whole batch).

WeakMap / WeakSet: leak-free by construction

`WeakMap` and `WeakSet` hold their keys **weakly** — a weak reference does *not* count for reachability. If the only thing pointing at a key object is a `WeakMap`, the collector is free to reclaim that object, and the entry disappears with it. This makes them the right tool for **caches and side-metadata keyed by an object you don't own the lifetime of**.

JS Copy

```
const meta = new WeakMap();
let node = { id: 1 };
meta.set(node, { visits: 5 }); // attach metadata without owning `node`
console.log(meta.get(node).visits); // ⇒ 5
console.log(meta.has(node));        // ⇒ true
// when `node` is set to null and nothing else references it,
// the WeakMap entry becomes collectable automatically.
```

Contrast with a `Map`: `map.set(node, ...)` is a *strong* reference, so `node` can never be collected while the map lives — exactly the "growing cache" leak. The `WeakMap` version self-cleans. The trade-offs are deliberate: `WeakMap` / `WeakSet` keys must be objects (not primitives), and they are **not enumerable** — no `.size`, no iteration, no `.keys()`. That is intentional: if you could enumerate them, the entries would be observable and their timing would leak GC behavior.

Typical uses: attaching private/derived data to DOM nodes or class instances, memoizing by object identity, and marking "have I already seen this object?" with a `WeakSet`.

`WeakRef` gives you a weak reference to a *single* object you can try to read via `.deref()` — it returns the object if still alive, or `undefined` if it has been collected. `FinalizationRegistry` lets you register a cleanup callback that *may* run after an object is collected.

JS Copy

```
let cache = { data: 42 };
const ref = new WeakRef(cache);
console.log(ref.deref().data); // ⇒ 42  (still reachable)

const reg = new FinalizationRegistry((label) ⇒ {
  // may run sometime AFTER the object is collected — timing NOT guarantee
});
reg.register(cache, 'cache-entry');
console.log(typeof ref.deref()); // ⇒ object
```

The spec is explicit that finalization timing is **non-deterministic** and callbacks may never run at all (e.g. on program exit). MDN and V8 both warn against relying on them for anything correctness-critical. Treat them as a last resort for advanced caching, and never for releasing important resources — use explicit lifecycle (`close()`, `dispose()`, `AbortController`) instead.

Spotting leaks in practice

You confirm a leak by measurement, not by staring at code. In Chrome DevTools:

- **Memory → Heap snapshot**: take a snapshot, exercise the suspect flow (e.g. open and close a modal 10 times), take a second snapshot, and use **"Comparison"** view to see what grew. Sort by delta / retained size.
- **Detached nodes**: filter a snapshot for `Detached` — detached `HTMLDivElement` entries are DOM leaks. The **retainers** pane shows the reference chain keeping each one alive, which points straight at the offending closure/array.

- Allocation timeline / "Record allocations": shows memory climbing over time; a healthy app returns to baseline after GC (sawtooth), a leaking one trends upward.
- In **Node**, run with `--inspect` and take heap snapshots in DevTools, or use `process.memoryUsage()` / `--max-old-space-size` and watch for steady growth across identical requests.

Prevention rules of thumb

- Prefer `const` / `let` and strict mode / modules — kill accidental globals.
- Every subscription has an unsubscription: pair `setInterval` ↔ `clearInterval`, `addEventListener` ↔ `removeEventListener` (or `AbortController`), timers with cleanup.
- Null out references you are done with, especially long-lived arrays/objects and DOM references.
- Bound your caches (max size + eviction, or TTL); reach for `WeakMap` / `WeakSet` when an object's own lifetime should drive eviction.
- Capture the minimum in closures — extract primitives instead of retaining big objects.
- Measure with heap snapshots before and after; don't guess.

Common interview questions

Q: How does garbage collection work in JavaScript?

Engines use **mark-and-sweep** based on **reachability**. Starting from roots (the global object, the current call stack, and closures), the collector *marks* every object reachable by following references, then *sweeps* (frees) everything unmarked. An object is kept alive if and only if it is reachable from a root — otherwise it is collected. V8 layers a generational, incremental collector on top for performance, but the core is mark-and-sweep.

Q: Name common causes of memory leaks.

Accidental globals; forgotten `setInterval` / `setTimeout`; detached DOM nodes still referenced from JS; closures capturing large data; unbounded caches (a `Map` /object

Q: Why use a `WeakMap` instead of a `Map`?

A `Map` holds its keys **strongly**, so any object used as a key can never be collected while the map lives — using it as a cache leaks. A `WeakMap` holds keys **weakly**: if nothing else references the key, the entry is collected automatically. Use `WeakMap` for metadata/caches keyed by objects (DOM nodes, instances) whose lifetime you don't control. The cost: keys must be objects and the collection is not enumerable (no `.size`, no iteration).

Q: Does mark-and-sweep handle circular references?

Yes. It only asks "is this reachable from a root?", so two objects that reference each other but are unreachable from any root are never marked and get collected. Cycles only defeat pure **reference counting**, which JS engines do not rely on.

Q: What is a detached DOM node?

A DOM element removed from the document tree but still referenced by JavaScript (a variable, array, or closure). Because JS keeps it reachable, the engine cannot free it or its subtree, so it accumulates. You find them via the `Detached` filter in a heap snapshot and fix them by dropping the JS references.

Gotchas / traps

Setting a variable to `null` frees nothing if another reference remains. GC is about reachability, not about any one variable. Nulling `o1` does nothing while `o2` still points at the same object.

JS Copy

```
let o1 = { big: new Array(1000) };
let o2 = o1;
o1 = null;          // object still reachable via o2 → NOT collected
console.log(o2.big.length); // ⇒ 1000
```

Closures capture the scope, not just the variables you reference. In many engines the whole variable environment is retained, so a closure that uses one small field can pin a

large sibling object alive. Extract what you need.

Anonymous listeners/timers can't be removed. `element.addEventListener('click', () => { ... })` gives you no handle to pass to `removeEventListener`, so it leaks on re-mount. Always keep a named reference (or use an `AbortController` signal).

`WeakMap` has no `.size` and isn't iterable — on purpose. If you find yourself wanting to enumerate a `WeakMap`, you want a `Map` (and the eviction responsibility that comes with it). Weak collections are deliberately opaque so GC timing stays unobservable.

You cannot force or observe GC from normal JS. There is no standard `gc()`; `FinalizationRegistry` callbacks are non-deterministic and may never fire. Never build correctness on collection timing — release resources explicitly.

A leak is not always growth you can see instantly. Generational GC collects short-lived objects cheaply, so small leaks hide until the old generation fills. Confirm with a *comparison* of two heap snapshots across a repeated action, not a single reading.

QUICK CHECK — no marks, just for you.

1. Two objects reference each other but nothing reachable from a root points at either of them. How does a modern JavaScript engine treat them, and why?

- ○ They leak, because a reference-counting collector keeps each object's count at 1 forever
- ○ They are collected only after you manually break the cycle by nulling one of the references
- ○ They are collected, because mark-and-sweep only asks whether an object is reachable from a root — cycles are never marked, so they are swept
- ○ They survive until program exit, because the engine cannot safely reclaim objects that still have any incoming reference

Check answer

2. ... structure for caching metadata keyed by DOM nodes or object instances whose lifetime you don't control?

- A WeakMap holds its keys weakly — a weak reference does not count for reachability, so once nothing else references a key object the entry is reclaimed automatically, whereas a Map's strong keys leak
- A WeakMap evicts the least-recently-used entry once it exceeds a fixed maximum size, giving you built-in LRU behavior
- A WeakMap lets you enumerate its entries and read its .size, so you can prune stale keys yourself on a timer
- A WeakMap can use primitive keys such as strings and numbers, which a Map cannot

Check answer

3. What does this snippet log?

JAVASCRIPT Copy

```
let o1 = { big: new Array(3).fill('x') };
let o2 = o1;
o1 = null;              // object still reachable via o2
console.log(o2.big.length);
```

- undefined
- 3
- TypeError: Cannot read properties of null (reading 'big')
- 0

Check answer

Previous
Symbols, Iterators & Proxy

LEARN MORE

ARTICLES & DOCS

Memory management (MDN)
MDN

VIDEOS



Understand the JS garbage collector in 4 min
YouTube



Memory management (mark & sweep)
YouTube



Heap, stack & garbage collection
YouTube

Hand-picked references for this lesson.